

UNIT - 5 Time series in R

- * Business today relies, profoundly on timely, informed decisions.
- * Analysing time series data helps them to analyse &
 - predict sales nos for the next fiscal year,
 - predict & take proactive measures to deal with overwhelming website traffic,
 - monitor competition position & much more.
- * several methods have evolved over time-based data that help with prediction & forecasting.
 - one such method, which makes use of time-based data is time-series modelling.
- * Time series modelling involves working on time (years, days, hours & minutes) based data, to derive hidden insights, which then lead to informed decision making.

⇒ Questions related to time series data :

- is there a trend ? → Do the measurements tend to increase or decrease over time?
- is there seasonality ? → Does the data regularly exhibit repeating pattern of highs & lows related to calendar time such as seasons, quarters, months & days of the week & soon..

- are there outliers in the data ?
- is there variance over time?
- Are there abrupt changes ?

* Uses of Time series data :-

- Trend Analysis
- Cyclical fluctuation Analysis
- Variance Analysis

* What is Time series data ?

- Time series analysis plays a major role in business analytics.
- It can be defined as quantities that trace the values taken by a variable over a period such as month, quarter or year.

Ex's: share market: prices of shares changes every second.

- measuring the level of unemployment each month of the year.

Types :

- univariate → when time series data uses a single quantity for describing values is termed Univariate
- multivariate
↳ when time series data uses more than a single quantity for describing values is called multivariate

②

* Time series analysis performs the analysis of both types of time series data.

* Basic R commands for Data visualization

→ R lang. provides many commands that plot the given data such as `plot()`, `hist()`, `pie()`, `boxplot()`, `stripchart()`, `curve()`, `abline()`, `qqnorm()` etc...

→ Among these `plot()` & `hist()` commands are mostly used in time series analysis.

* `plot()` function

* `plot()` command of R helps to create different types of charts. It has many options for visualizing data in ~~diff~~ different forms.

→ graphical parameters such as `col`, `font`, `lwd`, `lty`, `cex`, can also use the `plot` command for enhancing the visualization of time series data.

Basic syntax of `plot()`

`plot(x, y, type, main, sub, xlab, ylab, ...)`

where $x \rightarrow$ argument defines the co-ordinates of points that can be any R obj

$y \rightarrow$ argument is optional that contains y coordinates

type → argument defines which type of plot is to be drawn.

main → argument defines the title of the plot.

sub → defines subtitle of the plot.

xlab → defines title of x-axis

ylab → defines title of y-axis

⇒ values of "type" argument

value	Graph type
-------	------------

P → Points on type

L → Lines on plot

b → Both points & lines

c → For the lines part alone of b.

O → overplotting of lines & points

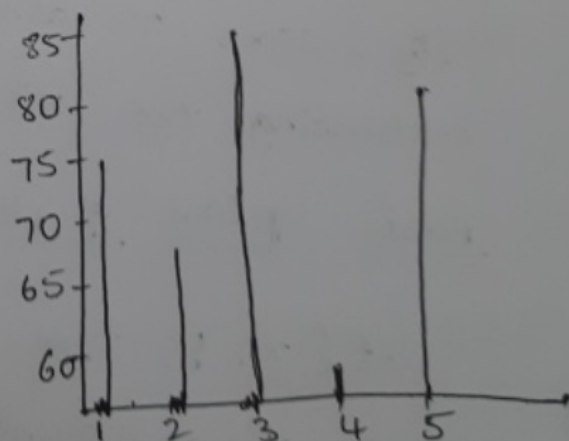
h → Histogram like vertical lines on plot.

s → stairs steps on plot.

S → other steps on plot

n → NO plotting.

Histogram



Ex

```
> S <- c(75, 68, 85, 57, 80)
```

```
> S
```

```
[1] 75 68 85 57 80
```

```
> plot(S)
```

```
> plot(S, type = "h", main = "Histogram", lwd = 4, col = "black")
```

* hist() function

- `hist()` command is used for creating the histogram of any data set.
- It uses different bars for the graphical representation of a dataset.
- It divides data set into certain ranges & creates bars of different heights.
- ⇒ Basic syntax of `hist()` fn: is

`hist(x, ....)`

where, `x` - is a vector of values ^{which} for the histogram is to be drawn.

Ex: # Reading a table `stuAt.csv` & plotting histogram

```
> s <- read.csv("stuAt.csv")
```

```
> s
```

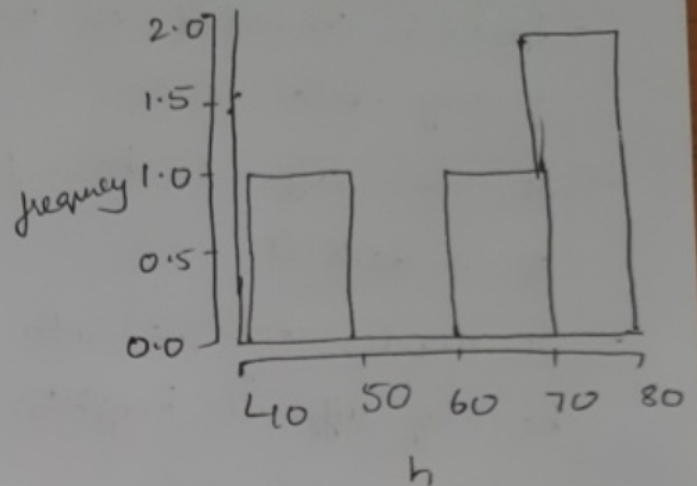
	sname	Jan	Feb	Mar	...
1	stu1	75	80	68	
2	stu2	45	56	69	
3	stu3	80	76	65	
4	stu4	67	45	46	

```
> h <- s$Jan
```

```
> h
```

[1]	75	45	80	67
-----	----	----	----	----

```
> hist(h, main = "Histogram", col = "grey")
```



* pie() function

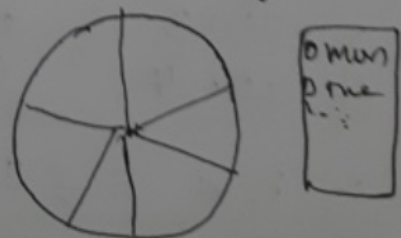
Syntax

```
pie(x, labels = names(x), edges = 200, radius = 0.8,
    clockwise = FALSE, init.angle = if(clockwise) 90 else 0,
    density = NULL, angel = 45,
```

- * $x \rightarrow$ vector of non-numerical values,
- * clockwise indicates if the slices are drawn clockwise or counter clockwise.

Adding legend

```
> legend("topright", c("Mon", "Tue", ...), cex = 0.8,
        fill = cols)
```



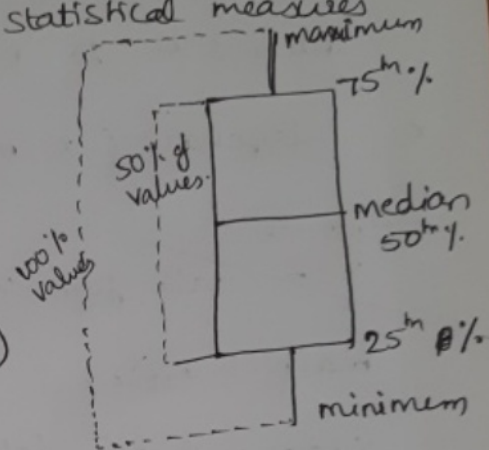
(4)

* boxplot() function

↳ Also referred to as box & whiskers plot.

It efficiently displays the 5 statistical measures

- minimum value
- lower quartile (25th %)
- median (50th %)
- upper quartile (75th %)
- maximum value (high value)



⇒ Advantages

- It can be drawn vertically or horizontally.
- provides a fair idea about the data's symmetry & skewness (skewness is asymmetry in statistical distribution).
- It shows outliers
- It allows easy comparison of datasets.

Syntax

`boxplot(x, ..., range = 1.5, width = NULL, yaxwidth = FALSE, ...)`

where $x \rightarrow$ is a numeric vector or a single list containing such vectors.

* stripchart() -fn: helps to create one-dimensional scatter plots (or dot plots) of the given data

→ when sample sizes are small, these plots are good alternative to boxplots.

⇒ stripchart(x, method="overplot", jitter=0.1, offset=1/3, vertical=FALSE, ...)

where,

x = data from which the plots are to be produced.

main = title.

xlab = x axis label.

ylab = y axis label.

method = method used to separate coincident points.

overplot - point will be overplotted.

jitter - jitter the points

stack - coincident points will be stacked.

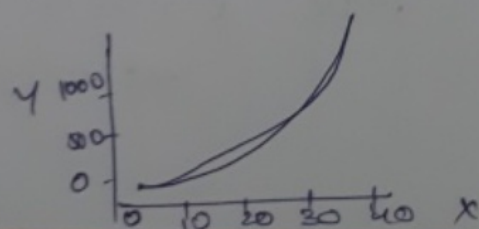
ex → stripchart(airqual\$ozone,
+ main = "mean ozone in facts ..",
+ xlab = " ",
+ ylab = " ",
+ method = "jitter", col = "orange")

* curve() -fn: : draws a curve corresponding to a fn: over the interval [from, to]. curve() can also plot an expression.

Syntax.

curve(expr, from=NULL, to=NULL, n=101, ...)

ex → curve(x^2, from=1, to=50, xlab="x", ylab="y")



* Basic 'R' commands for Data Manipulation

Time series analysis most often requires arithmetic mean, standard deviation, difference, probability distribution, density ...

⇒ Various commands that perform these operations & help manipulate time series data.

- `mean(x)` → ^{returns} arithmetic mean of given object.
- `diff(x)` → returns the lagged & iterated difference
- `sd(x)` → returns standard deviation of given obj
- `log(x)` → returns the logarithm of given obj
- `pnorm(x)` → returns the normal distribution fn.
- `dnorm(x)` → returns density of the obj

* Linear filtering of Time series

⇒ A simple component analysis divides the data into 4 main components :-

- trend
- seasonal
- cyclical
- irregular.

⇒ Each component has its own special feature.

i.e. • trend → long term progression,

• Seasonal → seasonal variation

• Cyclical → repeated but non periodic fluctuations

• irregular → random or irregular components of any time series.

→ The simple process of linear filtering converts the time series input data in linear OP to traditional time series analysis.

* Different classes of linear filters are available.

⇒ The moving averages with equal wts are a simple class of a linear filter.

It is defined as:

$$T_t = \frac{1}{2a+1} \sum_{i=-a}^a x_{t+i}$$

where,

T_t - trend component

x_t - any time series

a - moving average

⇒ "R" language provides the filter() function for linear filtering on time series analysis.

6

⇒ The `filter()` function generates the time series object or series of any given univariate time series or multivariate time series.

⇒ Basic syntax of `filter()` fn:

`filter(x, filter, method, ...)`

where `x` → contains univariate or multivariate time series.

`filter` → contains a vector of filter coefficients in reverse time order

`method` → defines a method for the linear filtering process.

- It can be either convolution (for moving average (MA))

or recursive (for auto regression (AR))

* Reading Time Series Data

R provides functions `scan()` & `tsc()` to read & store time series data into objects.

1) `scan()` fn:

`scan()` fn: reads the data from any file, as the time series data contains data with respect to successive time interval.

Syntax : scan (filename)

filename contains the name of the file to be read.

ii) ts() fn:

ts() fn stores the time series data & creates the time series object.

⇒ as.ts() → converts a simple object into a time series object.

⇒ is.ts() → checks whether an object is a time series object or not.

⇒ Syntax :

ts (data, start, end, frequency, class, ...)

where,

- data - contains time series values stored in any vector or matrix
- start - contains a single number or a vector of two integers that defines time of first observation.
- end - contains a single no: or a vector of two integers that define the time of the last observation.
- frequency - contains a single no: that defines the ~~class~~ no: of observations per unit of time.
- class - optional arg. that defines class for the o/p.

⑦

→ default class is "ts" for single series
 "mts", "ts", "matrix" etc.. are used for multiple classes.

Ex:

```
> S <- scan("Attendance.txt")
# Reads 30 items from the text file.
```

```
> S
```

```
[1] 45 56 60 35 . . . . .
```

```
> Sa <- ts(S)
```

```
> Sa
```

Time Series :

Start = 1

End = 30

Frequency = 1

```
[1] 45 56 60 35 . . . . .
```

⇒ Time series analysis also stores daily, monthly, quarterly or yearly data.
 we use frequency argument of ts() fn. for this.

Ex

```
> S <- scan("SA.dat")
```

```
# reads 24 items
```

```
> S
```

```
[1] 75 80 68 89 79 60 45 50 69 50 70
    55 88 76 65 78 85 88 67 45 46
    78 65 74.
```

Now storing this data in yearly format

```
> t <- ts(s, frequency=12, start=c(2011,1),
           end=c(2012,12))
```

```
> t
```

	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	...
2011	75	80	68	89	76	60	45	50	...
2012	80	76	65	78	85	88	67	45	...

(12th value)

```
> # Now storing the same data into quarterly format
```

```
> t <- ts(s, frequency=4, start=c(2011,1))
```

```
> t
```

	Qtr1	Qtr2	Qtr3	Qtr4
2011	75	80	68	89
2012	76	60	45	56
2013	69	50	70	55
2014	80	76	65	78
2015	85	88	67	45
2016	46	78	65	74

* Plotting Time series Data

An plotting task represents time series data graphically that is easily understandable by anyone.

⇒ plot() fn: is used for plotting time series Data.

Syntax : plot.ts(x)

x → any time series object.

Ex

```
> S <- scan("SA.dat")
> S
```

```
[1] 75 80 - - - - -
```

```
> t <- ts(S)
```

```
> t
```

```
Time Series:
```

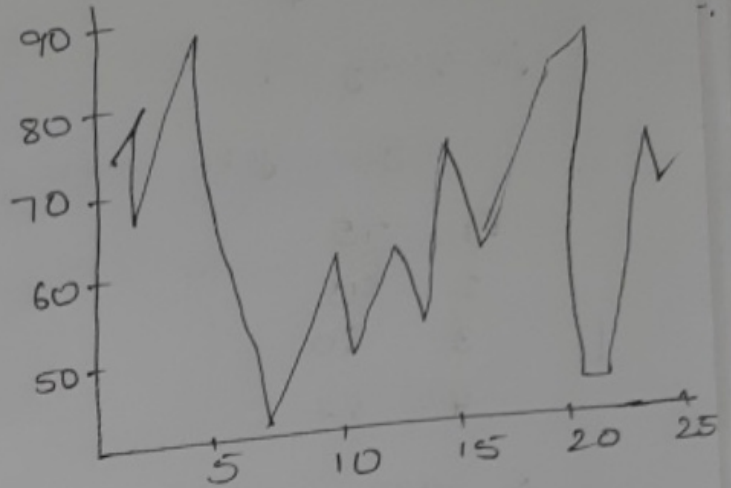
```
start = 1
```

```
end = 24
```

```
frequency = 1
```

```
[1] 75 80 - - - - -
```

```
> plot.ts(t) # plotting this data
```

Ex

following ex: creates plots of the attendance data of some students over six months.

```
> x <- read.csv("stuAt.csv")
```

```
> x
```

sname	Jan	feb	mar	Apr	may	june
1 S1	75	80	-	-	-	-
2 S2	45	56	-	-	-	-
3 S3	80	76	-	-	-	-
4 S4	67	45	-	-	-	-

```
> s <- x[, 2:7]
```

```
> s
```

```
> t <- ts(s, start = c(1, 1))
```

```
> t
```

```
plot
```

Time series :

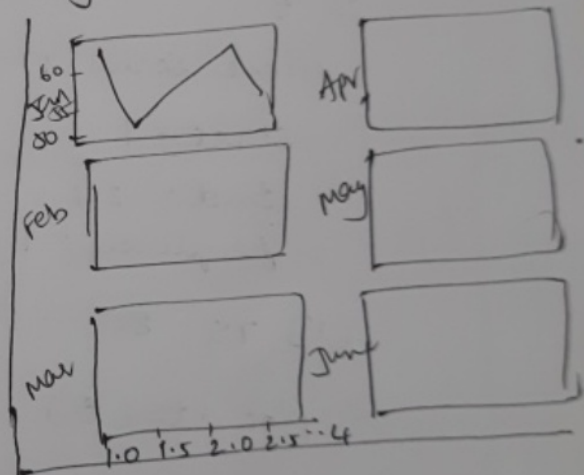
Start = 1

End = 4

frequency = 1

	Jan	Feb	Mar	Apr	May	June
0 ₁	75	-	-	-	-	-
2	45	/				
3	80	/				
4	67	/				

> plot.ts(t)



* Plotting Time series Data

For plotting time series data, R provides various fns, among which `plot()` ~~and~~ `fns` is the best.

Syntax: `plot.ts(x)`

where `x` is any time series object.

* Decomposing Time Series Data

- decomposing time series data is also a part of the simple component analysis that defines 4 components i.e. trend, seasonal, cyclical & irregular.
- time series changes because of seasonal, cyclical or irregular events so we need generate these components.
- The seasonal component contains the data that occurs seasonally every year.
ex fruit prices change according to season.
- The cyclical component contains data that changes ~~every~~ daily, weekly, monthly or annually.
ex share prices changes daily.
- The irregular component contains data that ~~changes~~ occurred at a specific time point but was not related to a season or cycle.
ex any natural event or political event that happens at a specific time.

- Decomposing time series data refers to a process that decomposes the given time series ^{data} into different components.
- In Business Analytics, decomposing is used to find out a particular component of seasonal or non-seasonal time series data.

* Decomposing Non-seasonal Data

- A non-seasonal time series contains the trends & the irregular components, so the decomposition process converts the non-seasonal data into these components.
- An additive model is used for finding out these components of the non-seasonal time series.
- This model uses a smoothing method by calculating the moving average of the time series.
- R provides a fn: $SMA()$ that smooths time series data by calculating moving average & estimates trend & irregular component.
- ⇒ "TTR" is the package that defines $SMA()$ fn:
 Syntax : $SMA(x, n, \dots)$
 where, $x \rightarrow$ is argument contains time series data, volume etc.
 $n \rightarrow$ contains numeric value for cal. ~~over~~ average.

ex: The ex: below takes time series data from "SA.dat" as seen plotting it shows random fluctuations in attendance over time.

So we use SMA() to estimate trend component. The fn: smooths the data using a simple moving avg of order 4.

```
> library("TTR")
> s <- scan("SA.dat")
# Reads 24 items
> s
```

```
[1] 75 80 68 . . . . .
```

```
> t <- ts(s)
```

```
> t
```

Time series:

```
start = 1
```

```
end = 24
```

```
frequency = 1
```

```
[1] 75 80 68 . . . . . (fig 1)
```

```
> plot.ts(t)
```

```
> sm <- sma(t, n=4)
```

```
> sm
```

Time series:

```
start = 1
```

```
end = 24
```

```
freq = 1
```

```
[1] NA NA NA 78.00, 78.25, 73.25, 67.50 . . . . .
```

```
> plot.ts(sm) (fig 2)
```

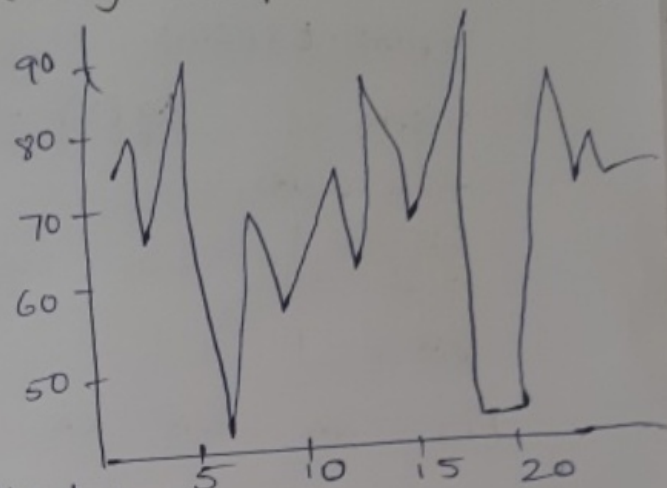


fig 1: time.

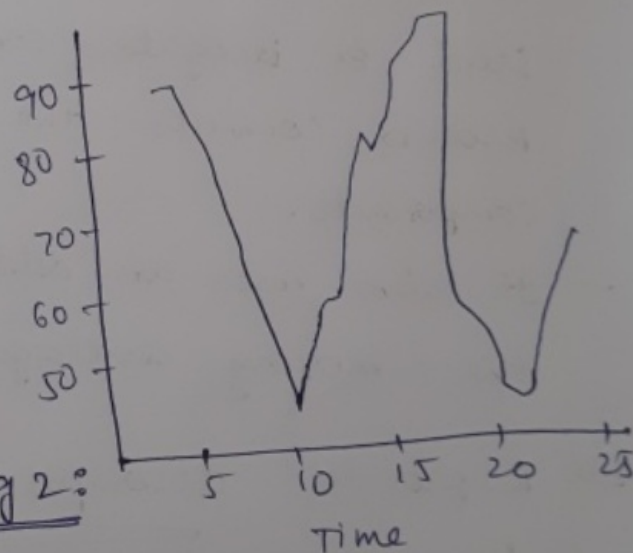


fig 2:

→ If the value of the order argument increases, it will plot more smoothed time series plot.

i.e higher the order value generates the trend component more accurately.

⇒ `> sm <- SMA(t, n=9)`

`> plot.ts(sm)`

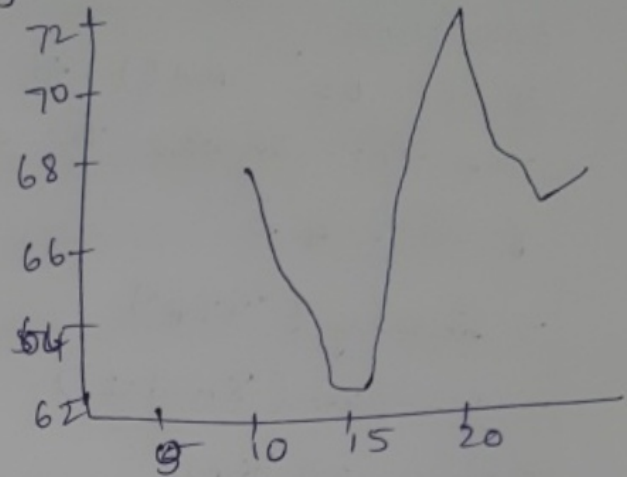
`> sm` fig(3) →

time series

start = 1

end = 24

freq = 1



• `[1] NA NA NA NA NA NA 68.6667, 65.8889,`
`64.778 - - - - -`

* Decomposing Seasonal Data

→ A seasonal time series contains the seasonal, trend & irregular components, so the decomposition process converts the seasonal data into these three components.

→ It also uses an additive model that calculates the moving average of time series & smooths the models.

⇒ R provides `decompose()` & `stl()` fn's for the decomposition of seasonal data.

i) decompose() fn:

→ It decomposes the time series data & also smooths time series data by cal. moving averages.

Syntax : `decompose(x, type, ...)`

where, $x \rightarrow$ contains time series y_{ij} for which components are to be estimated
 type - optional comp. that defines the type of component to be estimated.

ex

```
> x <- scan("SA.dat")
```

```
> t <- ts(x, freq=12)
```

```
> t
```

```
Jan Feb
```

```
1 75 80
```

```
2
```

```
> d <- decompose(t)
```

```
> d
```

```
$r
```

```
Jan Feb mar
```

```
1 75 80
```

```
2 80 7
```

```
$seasonal
```

```
Jan Feb
```

```
1 10.43
```

```
5.97
```

```
2 10.43
```

```
5.97
```

```
$ into irregular
```

```
$ trend
```

```
Jan Feb
```

```
[1]
```

ii) stl() fn

The seasonal trend decomposition (STL) is an algorithm that uses non-parametric regression method for calculating the components of R.

- R provides `stl()` for decomposing seasonal data using ~~loess~~ "loess" regression.
- It estimates the seasonal & trend components.

Syntax: `stl(x, s.window, ...)`

where $x \rightarrow$ contains time series object

$s.window \rightarrow$ contains either the character string "periodic" or numeric character

Ex `t <- ts(x, frequency = 4, start = c(2011, 1))`
`> t`
`atr1 atr2 ...`

2011

⋮

2016

`> d <- stl(t, s.window = "periodic")`

`> d`

call:

`stl(x = t, s.window = "periodic")`

Components

seasonal trend remainder

| - - - - -

Ex The following ex: creates a time series object 't' of the time series data stored in "SA.dat" file.

→ The `stl()` fn: returns all seasonal, trend & remainder components of this time series object 't' into a list objects.

→ All these components are graphically shown using the `plot()` fn:

```
> t <- ts(x, frequency = 4, start = c(2011, 1))
```

```
> t
```

	2011 Q1	2011 Q2	2011 Q3	2011 Q4
2011	75	76	69	80
2012	85	86	79	90
2013	95	96	89	100
2014	105	106	99	110
2015	115	116	109	120
2016	125	126	119	130

```
> d = stl(t, s.window = "periodic")
```

```
> d
```

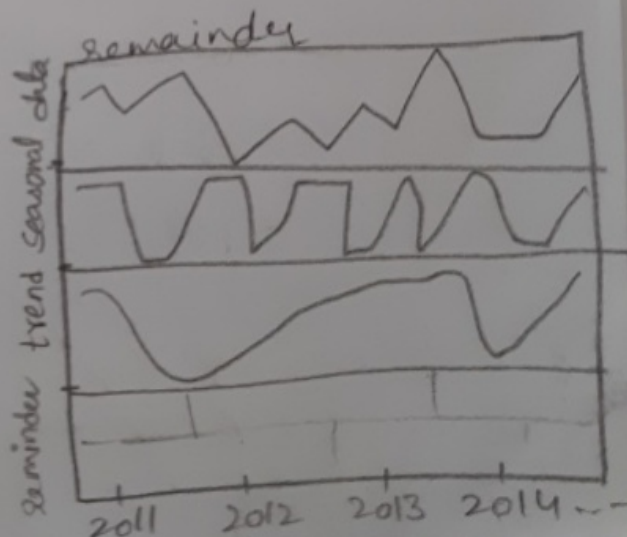
call:

```
stl(x = t, s.window = "periodic")
```

Components

```
2011 Q1
2011 Q2
2011 Q3
2011 Q4
```

seasonal trend



* Seasonal Adjustment

- A seasonally adjusted time series is a time series with no seasonality component.
- Simple method of calculating this series is to first calculate the seasonal component & then remove it from the original ~~series~~ time series.
- ⇒ This series provides trend component without any noise generated by the seasonality.

Ex:

```
> x <- scan("SA.dat")  
> t <- ts(x, frequency = 12)  
> d <- decompose(t) → decomposes return to  
components in an object 'd'  
> sa <- t - d$seasonal → calculates the  
seasonally adjusting  
component.  
> plot(sa)
```

(or)

```
> x  
> t t = ts ts(x, frequency = 4, start = 2011)  
> library("seasonal") → package seasonal.  
> se <- seas(t) → seas() is inbuilt fn:  
that finds seasonally  
adjusted series automatically  
> plot(se)
```

Forecasting using Exponential Smoothing

- Forecasts are a type of prediction that predict future events from past data.
- Here the forecast process uses exponential smoothing for making predictions.
- An exponential method finds out the changes in time series data by ignoring the irrelevant fluctuations & makes the short term forecast predictions for time series data.
- There are three types of exponential smoothing. They use a common inbuilt fn: HoltWinters() in R with different parameters.

Syntax :-

`HoltWinters(x, alpha = NULL, beta = NULL, gamma = NULL, ...)`

where, x - contains any time series object.

α , β , γ are parameters.

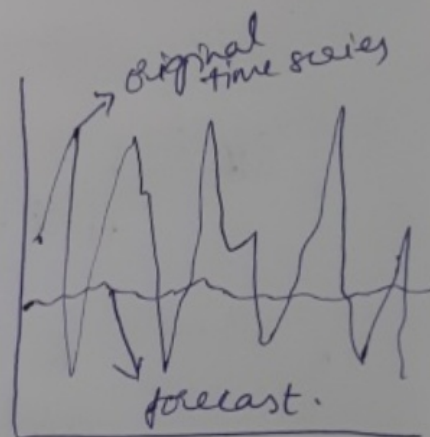
- ⇒ `HoltWinters()` fn: returns a value b/w 0-1 of all the three parameters (α , β & γ)
- ⇒ If value is near zero, it indicates that forecasts are done on less recent obs.
- ⇒ If value is near 1, then it indicates the forecasts are done on the most recent observation.

i) simple exponential smoothing

- A simple exponential smoothing estimates the level at the current time point & performs the short-term forecast.
- Alpha parameter of the `HoltWinters()` fn: controls the simple exponential smoothing.
so here to implement this type, beta + gamma parameters are set to FALSE in `HoltWinters()` fn:
- Here this `HoltWinters()` fn. implements exponential smoothing without any trend & seasonal components.

Ex :

```
> a <- scan("Attendance.txt")  
> a  
# reads attendance of 30 items  
> t <- ts(a) # creates timeseries object.  
> h <- HoltWinters(t, beta = FALSE,  
                  gamma = FALSE)  
> plot(h)  
> h.
```



HoltWinters exponential smoothing without trend & without seasonal comp.

call :

```
HoltWinters(x = t, beta = FALSE, gamma = FALSE)
```

smoothing parameters :

alpha = 0.030

beta = false

gamma = false

confidence

a 46.85

ii) Holt's Exponential Smoothing

⇒ It estimates the level & slope at the current time point.

→ The Alpha, Beta parameters control the Holt's exponential smoothing & estimate the level & slope resp.

⇒ It is the best method for timeseries containing trend component.

→ To impl. this type gamma is set to FALSE.

→ Here in this the values of alpha & beta are near to zero indicating forecasts are done on less recent obs.

Ex

```
> a <- scan("Attadace.txt")
```

```
> a
```

```
> t <- ts(a)
```

```
> h <- HoltWinters(t, gamma=FALSE)
```

```
> plot(h)
```

```
> h
```

H.W's exp. smoothing with trend & without seasonal comp

call
HoltWinters(x=t, gamma=FALSE)

alpha : 0.432...

beta : 0.368

gamma : FALSE

coefficients

a 47.4370

b 0.74368

ii) Holt's exponential smoothing

- ⇒ It estimates the level & slope at the current time point.
- The Alpha, Beta parameters control the Holt's exponential smoothing & estimate the level & slope resp.
- ⇒ It is the best method for timeseries containing trend component.
- To impl. this type gamma is set to FALSE.
- ⇒ Here in this the values of alpha & beta are near to zero indicating forecasts are done on less recent obs.

Ex > a <- scan("Attadace.txt")

> a

> t <- ts(a)

> h <- HoltWinters(t, gamma=FALSE)

> plot(h)

> h

H.W's exp. smoothing with trend & without seasonal comp

call
HoltWinters(x=t, gamma=FALSE)

alpha : 0.432...

beta : 0.368

gamma : FALSE

coefficients

a 47.4370

b 0.74368

iii) Holt-winters exponential smoothing

- ⇒ It estimates the level, slope & seasonal component at the current time point.
- ⇒ here all the three parameters controls the Holt-winters exponential smoothing & estimates the level, slope of trend component & seasonal component resp.
- * It is the best smoothing method for time series ~~data~~ containing trend & seasonal components.
- ⇒ To implement this fn: only timeseries obj need to be passed.
- ⇒ ^{gn} The below ex: all the parameters are zero, indicating that the forecasts are done on less recent observations in the time series.

Ex

```
> a # attendance data  
> t <- ts(a, frequency=12)  
> h <- HoltWinters(t)  
> h.
```

HoltWinters exp. Smoothing with trend & seasonal comp.

Call:

```
HoltWinters(x=t)  
smoothing param.  
alpha: 0  
beta: 0  
gamma: 0
```

here in this ex, both the original timeseries & forecast are almost same.

* ARIMA Models

(15)

↓
Autoregressive Integrated Moving Average

→ ARIMA models are another method of time series forecasting.

⇒ Exponential model makes short-term forecasts without using co-relation b/w the successive values of the time series. However sometimes a co-relation requires forecasting some irregular components.

→ In this case ARIMA models are best for forecasting.

⇒ The ARIMA model explicitly defines the irregular components of a stationary time series with non zero auto correlation.

dp: ⇒ ARIMA model is represented by $ARIMA(P, d, q)$
where $P \rightarrow$ autoregression (AR) order

$d \rightarrow$ degree of differencing

$q \rightarrow$ moving average (MA) order.

①

⇒ Important stages of ARIMA models are:-

i) Model estimation

ii) parameter checking

iii) forecasting

iv) Analysis of Autocorrelation

v) diagnostic checking.

②

i) Differencing a Time Series

→ This is the first step in finding an appropriate ARIMA model.

→ ARIMA models are basically used for a stationary time series.

So, for non-stationary time series it is necessary to difference the time series until a stationary time series is ~~not~~ obtained.

⇒ R provides `diff()` fn. for finding out the difference.

→ To obtain a stationary time series, it is necessary to pass the value of difference variable (d) of the ARIMA model in the `diff()` fn.

syntax:

`diff(x, differences...)`

where x → object to differentiate

differences → a numeric value that defines the order of difference ~~sections~~ ^{optimal} _{ags.}

ii) selecting a candidate ARIMA model

→ Next step in finding an appropriate ARIMA model is to analyse the autocorrelation and partial autocorrelation of the stationary time series.

⇒ This analysis helps to find the most appropriate values of P & q for an $ARIMA(P, d, q)$ model.

(16)

→ 'R' provides `acf()` & `pacf()` functions for finding the actual values of the autocorrelation (Q) & the partial autocorrelation (P) of the time series, resp.

Syntax

`acf(x, lag.max = NULL, ...)`

`Pacf(x, lag.max = NULL, ...)`

where, $x \rightarrow$ any time series obj or vector,
 $\text{lag.max} \rightarrow$ optional arg. that defines the maximum lag at which to calculate ACF or PACF

.... $\rightarrow$ optional arguments.

→ According to the value of autocorrelation & partial correlation, a suitable ARIMA model is selected.

→ R also provides an inbuilt function `auto.arima()` that automatically returns the best candidate model for given time series.

iii) Forecasting using an ARIMA model

→ After selecting the suitable candidate `ARIMA(P, d, q)` model, parameters of the selected `ARIMA(P, d, q)` model is found out.

→ These values are also used for defining the predictive model that makes the forecasts for the time series.

→ R provides arima() fn: to find out the parameters of the selected ARIMA(P, d, q) model.

Syntax arima(x, order(OL, OL, OL) ...)

where $x \rightarrow$ time series obj

order $\rightarrow$ defines the order components of ARIMA(P, d, q) model.

→ R also provides "forecast.Arima()" for making forecasting of the given time series.

It is available in "forecast" package.

iv) Analysis of Autocorrelations & partial Autocorrelations

→ Here values of auto correlation & partial correlation are analysed & simulated from an ARIMA(P, d, q) model.

→ R provides arima.sim() fn: that simulates these values.

Syntax :

arima.sim(model, n, ...)

where, model $\rightarrow$ contains list that defines the co-efficients of the component AR &/or MA.

V) Diagnostic checking

→ last step in fitting an ARIMA model proc.

$n \rightarrow$ optional arg. that contains a (+ve) value for defining the no. of ~~out~~^{put} series.

→ It checks the residuals from the fit model.

→ R provides tsdiag() fn: that creates a diagnostic plot for the time series fitted model.

Syn tsdiag(object, ...)
 $\hookrightarrow$ contains time series fitted model.

⇒ This function returns three plots i.e

- Standardised residuals plot.
- ACF of residuals plot
- plot defining the p-values of the Ljung-Box statistic.

Syntax:

tsdiag(Object, ---)

Ex of ARIMA Model

```
> x <- scan("Attendance.txt")
```

```
# Reads 30 items
```

```
> ax <- ts(x)
```

```
> ax
```

```
## Time series:
```

```
start: 1
```

```
end: 30
```

```
freq: 30
```

```
[1] values:-----
```

```
① > ad <- diff(ax, differences=2)
```

```
> plot.ts(ad)
```

```
> ad
```

```
## OP = -----
```

```
② > ac <- acf(ad)
```

```
> ac
```

```
## Autocorrelations of series 'ad' by lag.
```

```
> pac <- Pacf(ad)
```

```
> pac
```

```
## Partial autocorrelations of series 'ad' by lag
```

> ax <- ts(x)

> ax

dp: Timeseries ...

> # forecasting

> af <- arima(ax, order = c(0, 2, 1))

③ > af

call:
arima(=dp)

> library("forecast")

> f <- forecast.Arima(af)

> plot.forecast(f)

> f

④ > arima.sim(n=14, list(ar=c(1.000, -0.107),
ma=c(-0.5...))

dp:

> as <- arima.sim(n=14, list(ar=c(), ma())

> ts.plot(as)

> plot(as)

⑤ > af <- arima(ax, order = c(0, 2, 1))

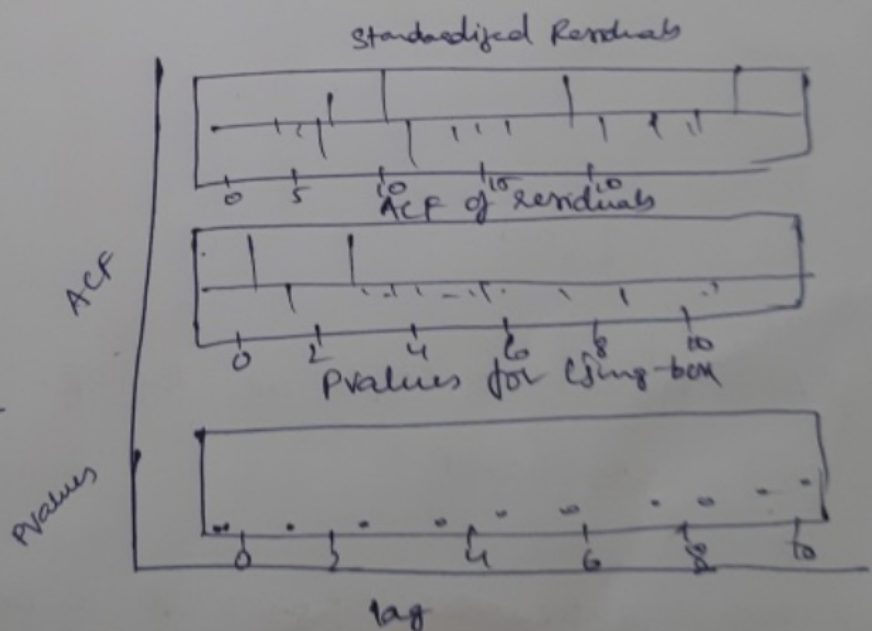
> af

dp:

> tsdiag(af)

final dp:

=====



fitted model.